SQL · POSTGRESQL

# Window Functions

Calculate across related rows without collapsing them. From OVER () to running totals, rankings and gaps-and-islands.

A guided tour, from simple to advanced



ROADMAP

# Four levels, each building on the last

LEVEL 1

## Foundations

OVER (), PARTITION BY, ROW_NUMBER, RANK and DENSE_RANK.

LEVEL 2

## Filtering

Where windows run in a query, and top-N per group.

LEVEL 3

## Frames and time

Running totals, moving averages, LAG and LEAD.

LEVEL 4

## Combining ideas

Shares of a total, windows over GROUP BY, islands, a capstone query.

THE PROBLEM

# GROUP BY collapses rows, windows keep them

### GROUP BY dept

`AVG(salary) per department`

| dept | n | avg_sal |
|---|---|---|
| Eng | 3 | 110000 |
| HR | 2 | 68500 |
| Sales | 3 | 81667 |

8 rows in, 3 rows out. The individual employees are gone.

### WINDOW FUNCTION

`AVG(salary) OVER (PARTITION BY dept)`

| name | dept | salary | dept_avg |
|---|---|---|---|
| Alice | Eng | 120000 | 110000 |
| Bob | Eng | 105000 | 110000 |
| Carla | Eng | 105000 | 110000 |
| Hugo | HR | 72000 | 68500 |
| Gina | HR | 65000 | 68500 |
| Eve | Sales | 95000 | 81667 |
| Dan | Sales | 80000 | 81667 |
| Frank | Sales | 70000 | 81667 |

8 rows in, 8 rows out. Each row carries its group’s average.

THE IDEA

# A window function calculates a value for each row from a set of related rows, and every row stays in the result.

### Rows in, rows out

The output has as many rows as the input. Nothing is collapsed.

### The window

The rows a given row is allowed to see. You define it inside OVER.

### Many functions

Aggregates (SUM, AVG), rankings (RANK) and lookups (LAG) all take OVER.

SYNTAX

# Anatomy of an OVER clause

```
SUM(revenue) OVER (
  PARTITION BY region
  ORDER BY month
  ROWS BETWEEN 2 PRECEDING
      AND CURRENT ROW
)
```

**Function**: what to calculate. SUM, AVG, RANK, LAG and more.

**PARTITION BY**: split rows into independent groups. Leave it out and there is one group.

**ORDER BY**: the order inside each group. Needed for ranking, LAG and running totals.

**Frame**: which nearby rows feed the function. Optional, and the default depends on ORDER BY.

SAMPLE DATA

# Two small tables for every example

`employees`

| name | dept | salary |
|---|---|---|
| Alice | Eng | 120000 |
| Bob | Eng | 105000 |
| Carla | Eng | 105000 |
| Dan | Sales | 80000 |
| Eve | Sales | 95000 |
| Frank | Sales | 70000 |
| Gina | HR | 65000 |
| Hugo | HR | 72000 |

`monthly_sales (12 rows, shown pivoted)`

| month | East | West |
|---|---|---|
| 2026-01 | 100 | 80 |
| 2026-02 | 120 | 70 |
| 2026-03 | 90 | 110 |
| 2026-04 | 150 | 130 |
| 2026-05 | 130 | 140 |
| 2026-06 | 170 | 160 |

Columns: region, month (a date), revenue in $K. One row per region per month.

LEVEL 1 · EXAMPLE 1

# Aggregate over every row: OVER ()

```
SELECT name, salary,
  ROUND(AVG(salary) OVER (), 0) AS avg_all
FROM employees;
```

Empty parentheses mean one window holding every row. The average appears beside each employee instead of replacing them.

| name | salary | avg_all |
|---|---|---|
| Alice | 120000 | 89000 |
| Bob | 105000 | 89000 |
| Carla | 105000 | 89000 |
| Dan | 80000 | 89000 |
| Eve | 95000 | 89000 |
| Frank | 70000 | 89000 |
| Gina | 65000 | 89000 |
| Hugo | 72000 | 89000 |

LEVEL 1 · EXAMPLE 2

# Split into groups: PARTITION BY

```
SELECT name, dept, salary,
  ROUND(AVG(salary)
    OVER (PARTITION BY dept)) AS dept_avg
FROM employees
ORDER BY dept, salary DESC, name;
```

> PARTITION BY is GROUP BY for windows: the same grouping, but no rows disappear.

| name | dept | salary | dept_avg |
|---|---|---|---|
| Alice | Eng | 120000 | 110000 |
| Bob | Eng | 105000 | 110000 |
| Carla | Eng | 105000 | 110000 |
| Hugo | HR | 72000 | 68500 |
| Gina | HR | 65000 | 68500 |
| Eve | Sales | 95000 | 81667 |
| Dan | Sales | 80000 | 81667 |
| Frank | Sales | 70000 | 81667 |

LEVEL 1 · EXAMPLE 3

# Number the rows: ROW_NUMBER()

```
SELECT name, salary,
  ROW_NUMBER()
    OVER (ORDER BY salary DESC, name) AS rn
FROM employees
ORDER BY rn;
```

ORDER BY inside OVER orders the window, not the output. Adding name breaks the tie between Bob and Carla.

| name | salary | rn |
|---|---|---|
| Alice | 120000 | 1 |
| Bob | 105000 | 2 |
| Carla | 105000 | 3 |
| Eve | 95000 | 4 |
| Dan | 80000 | 5 |
| Hugo | 72000 | 6 |
| Frank | 70000 | 7 |
| Gina | 65000 | 8 |

LEVEL 1 · EXAMPLE 4

# Handle ties: RANK and DENSE_RANK

```
SELECT name, salary,
  ROW_NUMBER() OVER w AS row_num,
  RANK()       OVER w AS rnk,
  DENSE_RANK() OVER w AS dense
FROM employees
WINDOW w AS (ORDER BY salary DESC)
ORDER BY salary DESC;
```

RANK skips after a tie (1, 2, 2, 4). DENSE_RANK does not (1, 2, 2, 3). WINDOW w names a window once so you can reuse it.

| name | salary | row_num | rnk | dense |
|---|---|---|---|---|
| Alice | 120000 | 1 | 1 | 1 |
| Bob | 105000 | 2 | 2 | 2 |
| Carla | 105000 | 3 | 2 | 2 |
| Eve | 95000 | 4 | 4 | 3 |
| Dan | 80000 | 5 | 5 | 4 |
| Hugo | 72000 | 6 | 6 | 5 |
| Frank | 70000 | 7 | 7 | 6 |
| Gina | 65000 | 8 | 8 | 7 |

Shaded rows tie on salary. Which of Bob and Carla gets 2 or 3 in row_num is arbitrary.

LEVEL 2 · EXECUTION ORDER

# Where windows run in a query

| 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|
| FROM | WHERE | GROUP BY | HAVING | **Window functions** | DISTINCT | ORDER BY | LIMIT |

### Filters run first

Rows removed by WHERE or HAVING never reach the window.

### No windows in WHERE

To filter on a rank or running total, wrap the query in a subquery or CTE.

### Groups first

GROUP BY runs before windows, so a window can work on grouped results.

LEVEL 2 · EXAMPLE 5

# Top N per group

```sql
SELECT name, dept, salary, rn
FROM (
  SELECT name, dept, salary,
    ROW_NUMBER() OVER (
      PARTITION BY dept
      ORDER BY salary DESC, name) AS rn
  FROM employees
) AS ranked
WHERE rn ⩽ 2
ORDER BY dept, rn;
```

| name | dept | salary | rn |
|---|---|---|---|
| Alice | Eng | 120000 | 1 |
| Bob | Eng | 105000 | 2 |
| Hugo | HR | 72000 | 1 |
| Gina | HR | 65000 | 2 |
| Eve | Sales | 95000 | 1 |
| Dan | Sales | 80000 | 2 |

Carla (third in Eng) and Frank (third in Sales) are filtered out.

> Rank in the inner query, filter in the outer one. Use RANK to keep ties.

LEVEL 3 · EXAMPLE 6

# Running totals: ORDER BY inside OVER

```
SELECT month, revenue,
  SUM(revenue)
    OVER (ORDER BY month) AS running
FROM monthly_sales
WHERE region = 'East';
```

With ORDER BY, each row sees everything from the first row up to itself. That turns SUM into a cumulative sum.

| month | revenue | running |
|---|---|---|
| 2026-01 | 100 | 100 |
| 2026-02 | 120 | 220 |
| 2026-03 | 90 | 310 |
| 2026-04 | 150 | 460 |
| 2026-05 | 130 | 590 |
| 2026-06 | 170 | 760 |

Revenue in $K. WHERE filters to East before the window runs.

LEVEL 3 · FRAMES

# The frame: which rows the function sees

### ROWS BETWEEN 2 PRECEDING AND CURRENT ROW

| Jan | Feb | Mar | Apr (current) | May | Jun |
|---|---|---|---|---|---|
| 100 | 120 | 90 | 150 | 130 | 170 |

### ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

| Jan | Feb | Mar | Apr (current) | May | Jun |
|---|---|---|---|---|---|
| 100 | 120 | 90 | 150 | 130 | 170 |

### ROWS BETWEEN 1 PRECEDING AND 1 FOLLOWING

| Jan | Feb | Mar | Apr (current) | May | Jun |
|---|---|---|---|---|---|
| 100 | 120 | 90 | 150 | 130 | 170 |

Teal boxes are in the frame for the Apr row. Default: no ORDER BY gives the whole partition; with ORDER BY the frame runs from the partition start to the current row.

LEVEL 3 · EXAMPLE 7

# Moving average with a ROWS frame

```
SELECT month, revenue,
  ROUND(AVG(revenue) OVER (
    ORDER BY month
    ROWS BETWEEN 2 PRECEDING
             AND CURRENT ROW), 1) AS ma3
FROM monthly_sales
WHERE region = 'East';
```

ROWS counts physical rows. The first two months average fewer than three values because the frame runs out of history.

| month | revenue | ma3 |
|---|---|---|
| 2026-01 | 100 | 100.0 |
| 2026-02 | 120 | 110.0 |
| 2026-03 | 90 | 103.3 |
| 2026-04 | 150 | 120.0 |
| 2026-05 | 130 | 123.3 |
| 2026-06 | 170 | 150.0 |

Apr: (120 + 90 + 150) / 3 = 120.0

LEVEL 3 · EXAMPLE 8

# Look at neighbours: LAG and LEAD

```sql
SELECT month, revenue,
  LAG(revenue)  OVER w AS prev,
  LEAD(revenue) OVER w AS next,
  revenue - LAG(revenue) OVER w AS chg
FROM monthly_sales
WHERE region = 'East'
WINDOW w AS (ORDER BY month);
```

LAG looks back, LEAD looks ahead. Missing neighbours give NULL, or a default: LAG(revenue, 1, 0).

| month | revenue | prev | next | chg |
|---|---|---|---|---|
| 2026-01 | 100 | NULL | 120 | NULL |
| 2026-02 | 120 | 100 | 90 | 20 |
| 2026-03 | 90 | 120 | 150 | -30 |
| 2026-04 | 150 | 90 | 130 | 60 |
| 2026-05 | 130 | 150 | 170 | -20 |
| 2026-06 | 170 | 130 | NULL | 40 |

LAG and LEAD ignore frames: they look at fixed offsets within the partition.

LEVEL 4 · EXAMPLE 9

# Share of the group: a window inside arithmetic

```
SELECT name, dept, salary,
  ROUND(100.0 * salary
    / SUM(salary) OVER (PARTITION BY dept),
    1) AS pct
FROM employees
ORDER BY dept, salary DESC, name;
```

A window aggregate is an ordinary expression. Use OVER () instead for percent of the company total.

| name | dept | salary | pct |
|---|---|---|---|
| Alice | Eng | 120000 | 36.4 |
| Bob | Eng | 105000 | 31.8 |
| Carla | Eng | 105000 | 31.8 |
| Hugo | HR | 72000 | 52.6 |
| Gina | HR | 65000 | 47.4 |
| Eve | Sales | 95000 | 38.8 |
| Dan | Sales | 80000 | 32.7 |
| Frank | Sales | 70000 | 28.6 |

pct is each salary as a percent of its department payroll.

LEVEL 4 · EXAMPLE 10

# The LAST_VALUE trap

```
SELECT name, dept, salary,
  LAST_VALUE(salary) OVER (
    PARTITION BY dept
    ORDER BY salary DESC) AS bad,
  LAST_VALUE(salary) OVER (
    PARTITION BY dept
    ORDER BY salary DESC
    ROWS BETWEEN UNBOUNDED PRECEDING
             AND UNBOUNDED FOLLOWING) AS good
FROM employees;
```

> The default frame ends at the current row. Widen it to reach the last row of the partition.

| name | dept | salary | bad | good |
|---|---|---|---|---|
| Alice | Eng | 120000 | 120000 | 105000 |
| Bob | Eng | 105000 | 105000 | 105000 |
| Carla | Eng | 105000 | 105000 | 105000 |
| Hugo | HR | 72000 | 72000 | 65000 |
| Gina | HR | 65000 | 65000 | 65000 |
| Eve | Sales | 95000 | 95000 | 70000 |
| Dan | Sales | 80000 | 80000 | 70000 |
| Frank | Sales | 70000 | 70000 | 70000 |

bad just echoes salary; good is the lowest salary in the department.

LEVEL 4 · EXAMPLE 11

# Windows over grouped results

```
SELECT dept,
  SUM(salary) AS payroll,
  RANK() OVER (ORDER BY SUM(salary) DESC)
    AS rnk,
  ROUND(100.0 * SUM(salary)
    / SUM(SUM(salary)) OVER (), 1) AS pct
FROM employees
GROUP BY dept
ORDER BY rnk;
```

| dept | payroll | rnk | pct |
|---|---|---|---|
| Eng | 330000 | 1 | 46.3 |
| Sales | 245000 | 2 | 34.4 |
| HR | 137000 | 3 | 19.2 |

**Read it in three steps**

1. GROUP BY collapses 8 employees into 3 department rows.
2. The window then runs over those 3 rows.
3. SUM(SUM(salary)) is the total of the department sums.

LEVEL 4 · EXAMPLE 12

# Gaps and islands: consecutive runs

```
SELECT MIN(d) AS first_day,
       MAX(d) AS last_day,
       COUNT(*) AS days
FROM (
  SELECT d,
    d - (ROW_NUMBER() OVER (ORDER BY d))::int
      AS grp
  FROM logins
) AS t
GROUP BY grp
ORDER BY first_day;
```

Days in the same run share the same date minus row number.

INNER QUERY (logins)

| d | rn | grp |
|---|---|---|
| 2026-03-01 | 1 | 2026-02-28 |
| 2026-03-02 | 2 | 2026-02-28 |
| 2026-03-03 | 3 | 2026-02-28 |
| 2026-03-05 | 4 | 2026-03-01 |
| 2026-03-06 | 5 | 2026-03-01 |
| 2026-03-09 | 6 | 2026-03-03 |

RESULT

| first_day | last_day | days |
|---|---|---|
| 2026-03-01 | 2026-03-03 | 3 |
| 2026-03-05 | 2026-03-06 | 2 |
| 2026-03-09 | 2026-03-09 | 1 |

LEVEL 4 · CAPSTONE

# Putting it together: a regional dashboard

```sql
WITH m AS (
  SELECT region, month, revenue,
    SUM(revenue) OVER w AS running,
    revenue - LAG(revenue) OVER w AS mom,
    ROUND(100.0 * revenue / SUM(revenue)
      OVER (PARTITION BY month), 1) AS share,
    RANK() OVER (PARTITION BY month
      ORDER BY revenue DESC) AS rnk
  FROM monthly_sales
  WINDOW w AS (PARTITION BY region
               ORDER BY month)
)
SELECT * FROM m
WHERE month ⩾ '2026-04-01'
ORDER BY month, rnk;
```

| region | month | revenue | running | mom | share | rnk |
|---|---|---|---|---|---|---|
| East | 2026-04 | 150 | 460 | 60 | 53.6 | 1 |
| West | 2026-04 | 130 | 390 | 20 | 46.4 | 2 |
| West | 2026-05 | 140 | 530 | 10 | 51.9 | 1 |
| East | 2026-05 | 130 | 590 | -20 | 48.1 | 2 |
| East | 2026-06 | 170 | 760 | 40 | 51.5 | 1 |
| West | 2026-06 | 160 | 690 | 20 | 48.5 | 2 |

- **running**: cumulative revenue per region
- **mom**: change since last month (LAG)
- **share**: the region's slice of the month
- **rnk**: which region led the month

WRAP-UP

# Common pitfalls

### Window in WHERE

Not allowed. Compute it in a subquery or CTE, then filter outside.

### Missing ORDER BY

ROW_NUMBER, LAG and running totals are arbitrary without one.

### Ties

Equal values make ROW_NUMBER unstable. Add a tie-breaker column.

### The default frame

With ORDER BY it stops at the current row. Spell out the frame for LAST_VALUE.

### ROUND on float

ROUND(x, n) needs numeric. Cast double precision values with ::numeric.

### Cost

Windows sort rows. An index on the PARTITION BY and ORDER BY columns can help; check EXPLAIN.

REFERENCE

# Window function cheat sheet

| Type | Function | Returns |
|---|---|---|
| Ranking | ROW_NUMBER() | Unique position 1, 2, 3 ... |
| Ranking | RANK(), DENSE_RANK() | Position with ties, with or without gaps |
| Ranking | NTILE(n) | Bucket number from 1 to n |
| Ranking | PERCENT_RANK(), CUME_DIST() | Relative position between 0 and 1 |
| Offset | LAG(x, n), LEAD(x, n) | Value n rows before or after |
| Offset | FIRST_VALUE, LAST_VALUE, NTH_VALUE | Value at a position in the frame |
| Aggregate | SUM, AVG, MIN, MAX, COUNT | Any aggregate, used with OVER |
| Frame | ROWS, RANGE, GROUPS | Count rows, value distance or peer groups |

YOUR TURN

# Practice exercises

**1 · EASY**

Show each employee beside the highest salary in the company.

**2 · EASY**

Number the employees within each department, alphabetically by name.

**3 · MEDIUM**

Show each employee’s gap to the top salary in their department.

**4 · MEDIUM**

For East, compute a rolling 3-month revenue total.

**5 · HARD**

For each region, find the month with the biggest increase over the previous month.

**6 · HARD**

Using logins, report the longest streak of consecutive days.

TAKEAWAYS

# Three things to remember

1. A window adds a calculation to every row. No row disappears.
2. Three controls shape it: PARTITION BY for groups, ORDER BY for sequence, a frame for nearby rows.
3. Windows run after WHERE and GROUP BY. To filter on one, wrap the query in a subquery or CTE.

# Questions?